$$[\text{Prog}] \to [\text{Stmt}]^* \tag{1}$$

$$[\text{Stmt}] \to \begin{cases} def[\text{Type}]\text{ident}[= [\text{Expr}]]^?; \\ def[\text{Type}][]\text{ident}[= [\text{Expr}]^?]; \\ func[\text{Type}]\text{ident}([\text{ident}]^*)[\text{Scope}] \\ \text{ident} = [\text{Expr}]; \\ \text{ident}[[\text{Expr}]] = [\text{Expr}]; \\ \text{ident}([\text{Expr}]^*); \\ \text{return}[\text{Expr}]^?; \\ [\text{Scope}] \\ if([\text{Expr}])[\text{Scope}][\text{IfPred}] \\ while([\text{Expr}])[\text{Scope}] \\ print([\text{Expr}]); \\ exit([\text{Expr}]); \end{cases} \tag{2}$$

$$[\text{Type}] \to \begin{cases} bool \\ int \end{cases} \tag{3}$$

$$[\text{Scope}] \to \{[\text{Stmt}^*]\} \tag{4}$$

$$[\text{IfPred}] \to \begin{cases} elseif([\text{Expr}])[\text{Scope}][\text{IfPred}] \\ else[\text{Scope}] \\ \epsilon \end{cases} \tag{5}$$

$$[\text{Expr}] \to \begin{cases} [\text{Atom}] \\ [\text{BinExpr}] \\ \text{ident}([\text{Expr}]*) \end{cases} \tag{6}$$

$$[\text{BinExpr}] \to \begin{cases} [\text{Expr}] * [\text{Expr}] & \text{prec} = 2 \\ [\text{Expr}]/[\text{Expr}] & \text{prec} = 2 \\ [\text{Expr}] - [\text{Expr}] & \text{prec} = 1 \\ [\text{Expr}] + [\text{Expr}] & \text{prec} = 1 \\ [\text{Expr}] == [\text{Expr}] & \text{prec} = 0 \\ [\text{Expr}]! = [\text{Expr}] & \text{prec} = 0 \\ [\text{Expr}] >= [\text{Expr}] & \text{prec} = 0 \\ [\text{Expr}] <= [\text{Expr}] & \text{prec} = 0 \\ [\text{Expr}] > [\text{Expr}] & \text{prec} = 0 \\ [\text{Expr}] < [\text{Expr}] & \text{prec} = 0 \end{cases} \tag{7}$$

$$[\text{Atom}] \to \begin{cases} \text{int\_lit} \\ \text{ident} \\ \text{ident}[[\text{Expr}]] \\ ([\text{Expr}]) \end{cases} \tag{8}$$

Next steps implement arrays implement strings
Note: for functional purity local vars are not accessible inside functions